

PRÁCTICA 2

ENTRADA/SALIDA DE PROPÓSITO GENERAL (GPIO). INTERRUPCIONES

Juan Pedro Bandera Rubio (jpbandera@uma.es)

Esta práctica cumple un doble objetivo: por una parte, en ella programarás entradas y salidas por los pines GPIO disponibles en tu ESP32-S3-DevKitC-1.

Por otro lado, también gestionarás interrupciones asociadas a los GPIO configurados como entradas. Esto te permitirá ver, por primera vez, cómo se trabaja con interrupciones en tu ESP32-S3.

Puedes descargar del CV de la asignatura o de nuestro GitHub el código de ejemplo *ESP32_S3_espidf_GPIO*. Como ves, a partir de esta práctica ya sólo se van a proporcionar los códigos en el framework ESP-IDF. También, como verás en el programa de ejemplo, los mensajes de traza y el chequeo de condiciones de error se harán exclusivamente con la librería de logging de ESP-IDF (*esp_log.h*).

NOTA: Puedes programar tus prácticas en el framework que quieras, bien Arduino bien espidf. Por lo general, eso no tendrá efecto en la nota final. También puedes usar *printf()* para sacar mensajes de traza, en lugar de *ESP_LOGx*, y no gestionar condiciones de error o gestionarlas a mano, en lugar de con la macro *ESP_ERROR_CHECK()*. Sin embargo, creemos que es importante que te familiarices con estas formas de desarrollar código para ESP32 de una forma más profesional.

Para ejecutar el programa de ejemplo es necesario que dispongas de una protoboard, un pulsador, dos diodos LED y dos resistencias de 220Ω, además de cables de conexión. El objetivo es conectar el pulsador a una entrada GPIO, y los diodos LED (con sus correspondientes resistencias de protección de 220Ω en serie) a dos salidas GPIO.

El código mostrará en uno de los LED el estado del pulsador (LED encendido cuando el botón está pulsado, LED apagado cuando no lo está). El otro LED cambiará de estado cada vez que se detecte un flanco de subida en la entrada GPIO donde está conectado el pulsador. Es importante configurar en el archivo *platformio.ini* qué GPIO vas a usar para cada cosa (por defecto no hay valores asignados así que te dará error al compilar).

ENTRADA/SALIDA DE PROPÓSITO GENERAL (GPIO)

Lo más importante: **recuerda que el ESP32-S3 funciona estrictamente a 3,3 voltios.**

La referencia para programar GPIO en el ESP32-S3 la puedes encontrar en este enlace:

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/gpio.html>

NOTA: Cuando consultes información en la documentación de Espressif (enlaces como el de arriba), ten cuidado de comprobar que estás viendo la **documentación para el ESP32-S3**, y no para otro miembro de la familia ESP32.

Ahí puedes ver información sobre el mapa de GPIO del micro, explicaciones sobre el funcionamiento, aplicaciones de ejemplo y la API.

Otra muy buena referencia sobre el mapa de GPIO del ESP32-S3-DevKitC es este enlace, aunque no es una página oficial del ESP32-S3: <https://lastminuteengineers.com/esp32-s3-devkitc-pinout-reference/>

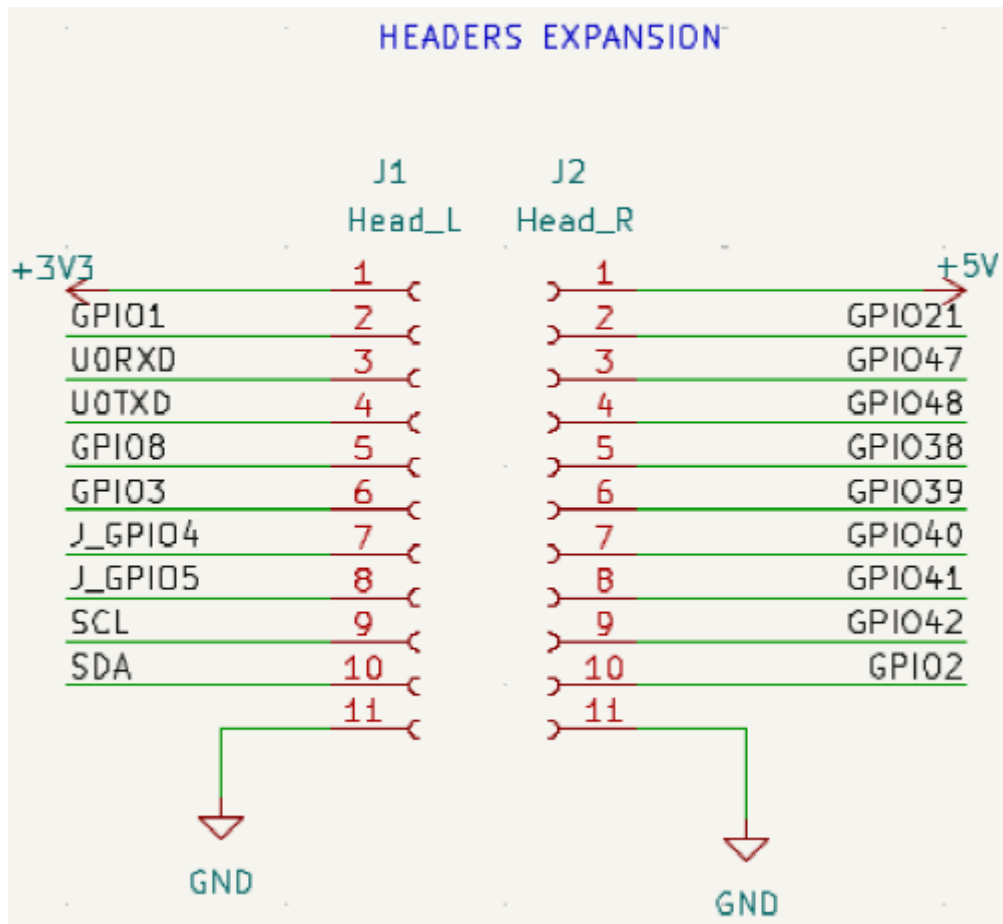
A continuación vamos a resaltar algunos de los aspectos más importantes sobre los GPIO del ESP32-S3, usando el código de ejemplo *ESP32_S3_espidf_GPIO* como guía. Pero ten en cuenta los enlaces anteriores, y por supuesto el datasheet y el manual de referencia técnica del ESP32-S3 para una información más completa.

MAPA DE PINES GPIO DE LA BREADBOARD DEL MICROBOT

No todos los 45 GPIO del ESP32-S3 quedan expuestos en la placa que estás usando. La ESP32-S3-DevKitC expone "sólo" 36 pines GPIO: del GPIO0 al GPIO21, y del GPIO35 al GPIO48. Además, cuando conectas la placa a la Breadboard del robot, la lista de pines expuestos se reduce aún más, ya que algunos se fijan para el control de motores, I2C, etc.

Si planeas usar la placa ESP32-S3-DevKitC para otra cosa que no sea esta asignatura, es necesario que consultes los enlaces web que se han indicado arriba para comprobar qué GPIO se pueden usar sin problemas, y cuáles deben usarse sólo sabiendo lo que se hace y con precaución.

Para esta asignatura, al conectar la placa a la Breadboard, se simplifica la situación. Los pines que quedan expuestos son los de los conectores de expansión de la placa, que se indican en la siguiente figura:



Puedes usar los GPIO indicados en el esquemático. En el caso de J_GPIO4 y J_GPIO5, según conectes el jumper correspondiente, puedes usarlos (a) como quieras; o (b) como GPIO de salida, conectados a los LED de la Breadboard (así, por ejemplo, te ahorrarías tener que montar los dos LED con las resistencias de protección del código de ejemplo).

CONFIGURACIÓN Y USO DE GPIO

Seguimos los pasos del código de ejemplo, dando por hecho que se ha dado valor a los GPIO en el archivo *platformio.ini*. En orden de ejecución se hace lo siguiente:

1.- Incluir el archivo de cabecera que necesitas para manejar los GPIO.

```
#include <driver/gpio.h>
```

2.- Seleccionar los pines como GPIO (en lugar de como pines de periféricos).

```
// configure GPIO pins as general purpose input/output
esp_rom_gpio_pad_select_gpio(LED_PIN);
esp_rom_gpio_pad_select_gpio(LED_PIN_2);
esp_rom_gpio_pad_select_gpio(INPUT_PIN);
```

3.- Configurar los GPIO conectados a LED como salidas.

```
// configure outputs
ESP_ERROR_CHECK(gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT));
ESP_ERROR_CHECK(gpio_set_direction(LED_PIN_2, GPIO_MODE_OUTPUT));
```

4.- Configurar el GPIO conectado al pulsador como entrada. Con resistencia de pull-up interna (el pulsador estará conectado por una pata a masa, por la otra al GPIO).

```
// configure input with pull-up
ESP_ERROR_CHECK(gpio_set_direction(INPUT_PIN, GPIO_MODE_INPUT));
ESP_ERROR_CHECK(gpio_pulldown_dis(INPUT_PIN));
ESP_ERROR_CHECK(gpio_pullup_en(INPUT_PIN));
```

INTERRUPCIONES

Si quieres simplemente sondear el estado de una entrada GPIO (hacer *polling*), con lo que acabamos de ver es suficiente. Pero lo más normal es que quieras que el microcontrolador trabaje por interrupciones. Aquí, de nuevo, vamos a dar una visión rápida de cómo se hace esto, basándonos en el código de ejemplo. Más información en la documentación de Espressif, en concreto en: https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/intr_alloc.html

La configuración de las interrupciones para una entrada GPIO se hace, en el código de ejemplo, en las siguientes instrucciones:

5.- Determinar el flanco del GPIO que provoca interrupción (recuerda: si tienes dudas sobre una función o variable, selecciónala y pulsa F12 para ir a su declaración. Podrá serte útil, por ejemplo, para ver los tipos de interrupción que puedes emplear).

```
// interrupt on rising edge
ESP_ERROR_CHECK(gpio_set_intr_type(INPUT_PIN, GPIO_INTR_POSEDGE));
```

6.- Crear una cola donde la interrupción del GPIO cargará el dato que veas oportuno, y una tarea que leerá esa cola cuando llegue algún dato (= se produzca una interrupción). Esta no es la forma obligatoria de gestionar las interrupciones, por supuesto. Tu rutina de interrupción podría hacer otra cosa, en vez de enviar un dato a una cola. Por ejemplo, cambiar el valor de una variable global.

```
// create a FreeRTOS queue to handle GPIO interrupts and a task to process them

interruptQueue = xQueueCreate(10, sizeof(int));
xTaskCreate(LED_Control_Task, "LED_Control_Task", 2048, NULL, 1, NULL);
```

7.- Instalar el driver deseado para manejar las interrupciones GPIO del ESP32-S3. Hay básicamente dos opciones: o la interrupción de cualquier GPIO activa una rutina de tratamiento de interrupción global (eso se hace empleando la función

gpio_install_isr_register), o se habilita la opción de tener rutinas de tratamiento de interrupción independientes para cada pin GPIO (eso se hace empleando la función *gpio_install_isr_service*). Esta última es la que usaremos.

```
// install the GPIO driver's ETS_GPIO_INTR_SOURCE ISR handler service, which
allows per-pin GPIO interrupt handlers (check API)
ESP_ERROR_CHECK(gpio_install_isr_service(0));
```

8.- Añadir el manejador de interrupciones para el pin GPIO.

```
// add the interrupt handler for the input pin
ESP_ERROR_CHECK(gpio_isr_handler_add(INPUT_PIN, gpio_interrupt_handler, (void
*)INPUT_PIN));
```

9.- Finalmente, habrá que programar el manejador de interrupciones del pin. En otras palabras, la rutina de tratamiento de interrupción (ISR) para ese pin GPIO. Fíjate cómo se emplea la macro *IRAM_ATTR*. Indica que esta rutina no se debe cargar en la memoria Flash (que es donde se carga el código por defecto), sino en la IRAM (RAM interna), que es donde deben guardarse las rutinas de manejo de interrupción en el ESP32-S3.

```
static void IRAM_ATTR gpio_interrupt_handler(void *args) {
int pinNumber = (int)args;
xQueueSendFromISR(interruptQueue, &pinNumber, NULL);
}
```

NOTA: Si hubieses escogido una rutina de interrupción global, cualquier interrupción de cualquier GPIO sería tratada por esa única rutina.

Si repasas ahora el código, verás cómo esta rutina ISR carga el valor del pin GPIO que ha provocado la interrupción en la cola de mensajes FreeRTOS. Entonces, la rutina *LED_Control_Task* sabe qué pin GPIO ha provocado una interrupción simplemente leyendo el valor que acaba de llegar a la cola.

ANTIRREBOTE SOFTWARE

Si ejecutas el código de ejemplo, verás que cada vez que le das al pulsador se están registrando, en realidad, varias interrupciones. Como ya sabes, esto es debido a los rebotes.

Para recordar qué son los rebotes: cada vez que le das al pulsador se produce un tren aleatorio de pulsos, que dura unos milisegundos. Lo típico es entre 1 y 20 milisegundos, pero eso depende muchísimo del pulsador, los cables, las conexiones... en otras palabras, si has conectado tu pulsador al microbot con cables dupont extra largos que tienen cierta holgura en las conexiones, puedes esperar tiempos de rebote bastante más largos.

Una forma de eliminar los rebotes es añadiendo a tu código un antirrebote software. Esto se hace de la siguiente manera:

- Cuando se produce una interrupción (¡por flanco!) en un GPIO, en la rutina ISR se *deshabilita* la interrupción de ese GPIO y se *habilita* un temporizador que cuenta unas decenas de milisegundos. El temporizador se configura como one-shot (una única cuenta). Si estás utilizando cola de mensajes para que la ISR comunique la interrupción, resetéala también.
- Cuando el temporizador llega a su final de cuenta, vuelve a habilitar las interrupciones por ese pin GPIO.

Con esto, durante todo el tiempo en que se pueden producir rebotes (en realidad, durante todo el tiempo de cuenta del temporizador) tendrás deshabilitadas las interrupciones en ese GPIO y, básicamente, ignorarás esas interrupciones espúreas.

PRÁCTICA EN CLASE: CREACIÓN DE UN ANTIRREBOTE SOFTWARE

Como ya has recordado cómo usar los timers de FreeRTOS (Práctica 1), y acabas de ver cómo gestionar los GPIO (Práctica 2), ahora tendrás que realizar una práctica en clase en la que añadirás un antirrebote software al código de ejemplo *ESP32_S3_espidf_GPIO*, para que las interrupciones del pulsador se gestionen adecuadamente.